SZABIST
University

Shaheed Zulfikar Ali Bhutto
Institute of Science and Technology

ROBOTICS & ARTIFICIAL INTELLIGENCE DEPARTMENT

Total Marks:_____

Obtained Marks: _____

Computing Vision

Lab Task # 10

Submitted To: **Mr. Muhammad Azhar**

Student Name: **Habiba Bibi**

Reg. Number: **23108111**

Code Example: Real-Time Object Detection for Smart Traffic Surveillance System

A municipal traffic department is deploying AI-powered surveillance cameras at major intersections in a metropolitan area. Their goal is to monitor real-time traffic flow, detect rule violations, and identify vehicles or pedestrians involved in incidents. Traditional manual review of camera footage is time-consuming and error-prone, leading to delays in response and enforcement.

Problem Statement:

Design and implement an automated object detection system using YOLOv8 to detect common objects such as cars, trucks, motorcycles, bicycles, pedestrians, and traffic signs in real-time footage captured by smart surveillance cameras.

Objective of the Lab Exercise:

- **Use a pre-trained YOLOv8 model to perform object detection on an input traffic image.**
- **Identify and draw bounding boxes around detected objects with confidence scores.**
- **Visualize the results using Matplotlib in a Jupyter/Google Colab environment.**

Tools Used:

- **Python**
- **OpenCV**
- **Matplotlib**
- **Ultralytics YOLOv8 Model (yolov8n.pt)**

```python
# Code Example: Real-Time Object Detection for Smart Traffic Surveillance System
from ultralytics import YOLO
import cv2
import matplotlib.pyplot as plt
model = YOLO('yolov8n.pt')
image_path = "D:/23108111/H19.jpg"
image = cv2.imread(image_path)
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
results = model(image_path)
results[0].plot()
plt.figure(figsize=(12, 8))
plt.imshow(results[0].plot()[..., ::-1])
plt.axis('off')
plt.title("Object Detection using YOLOv8")
plt.show()
```

```
image 1/1 D:\23108111\H19.jpg: 480x640 2 persons, 1 car, 3 buss, 1 truck, 85.0ms
Speed: 5.1ms preprocess, 85.0ms inference, 1.9ms postprocess per image at shape (1, 3, 480, 640)
```

Case Study 1: Real-Time Surveillance Analysis in Public Parks

A city's public safety department is piloting a smart surveillance system in public parks. The objective is to monitor foot traffic and detect the presence of individuals, animals, and suspicious activities in real-time. The system uses pre-trained YOLOv8 models to identify humans, animals, backpacks, bicycles, and more from live camera feeds or still images captured periodically.

This particular prototype tests detection capabilities using an image of a girl standing outdoors with background elements (trees, objects, etc.). The system highlights detected objects with bounding boxes to simulate how it would analyze a frame in real-time.

Objective:

Implement object detection on a surveillance image using a pre-trained YOLOv8 model, interpret detection results, and visualize identified entities to enhance park monitoring and safety systems.

```
# Case Study 1: Real-Time Surveillance Analysis in Public Parks
import cv2
from ultralytics import YOLO
model = YOLO("yolov8n.pt")
results = model("D:/23108111/H16.jpg")
import matplotlib.pyplot as plt
import cv2
img = results[0].plot()
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
plt.imshow(img_rgb)
plt.axis("off")
plt.show()
```

```
image 1/1 D:\23108111\H16.jpg: 640x416 1 person, 78.2ms
Speed: 3.4ms preprocess, 78.2ms inference, 1.2ms postprocess per image at shape (1, 3, 640, 416)
```

Object Detection Using SSD

Object detection detects instances of certain classes in digital images and videos (which can be regarded as sequences of images). In this notebook we demonstrate how to use pretrained Analytics Zoo model to detect objects in the video.

We used one of the videos in Youtube-8M (link) for demo, which is a scene of training a dog. We use our SSD-MobileNet model (downloadable from link) pretrained on PASCAL VOC dataset (link) for detection. It is able to detect 20 classes including person and dog. In the video the dogs and persons are are identified with boxes and class scores are shown on top.

References:

□ YouTube-8M: A Large and Diverse Labeled Video Dataset for Video Understanding Research (link).

```python
# Object Detection Using SSD
from ultralytics import YOLO
import cv2
from moviepy.editor import VideoFileClip
model = YOLO("yolov8n.pt")
video_path = "D:/23108111/Lecture H.mp4"
cap = cv2.VideoCapture(video_path)
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 640)
out = cv2.VideoWriter(
    "output.mp4",
    cv2.VideoWriter_fourcc(*'mp4v'),
    5,
    (640, 640)
)
while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break
    frame = cv2.resize(frame, (640, 640))
    results = model.predict(frame, imgsz=640, conf=0.5, verbose=False)
    annotated_frame = results[0].plot()
    out.write(annotated_frame)
cap.release()
out.release()
print("Video saved as output.mp4")
clip = VideoFileClip("output.mp4")
clip.write_gif("output.gif")
print("GIF saved as output.gif")
```

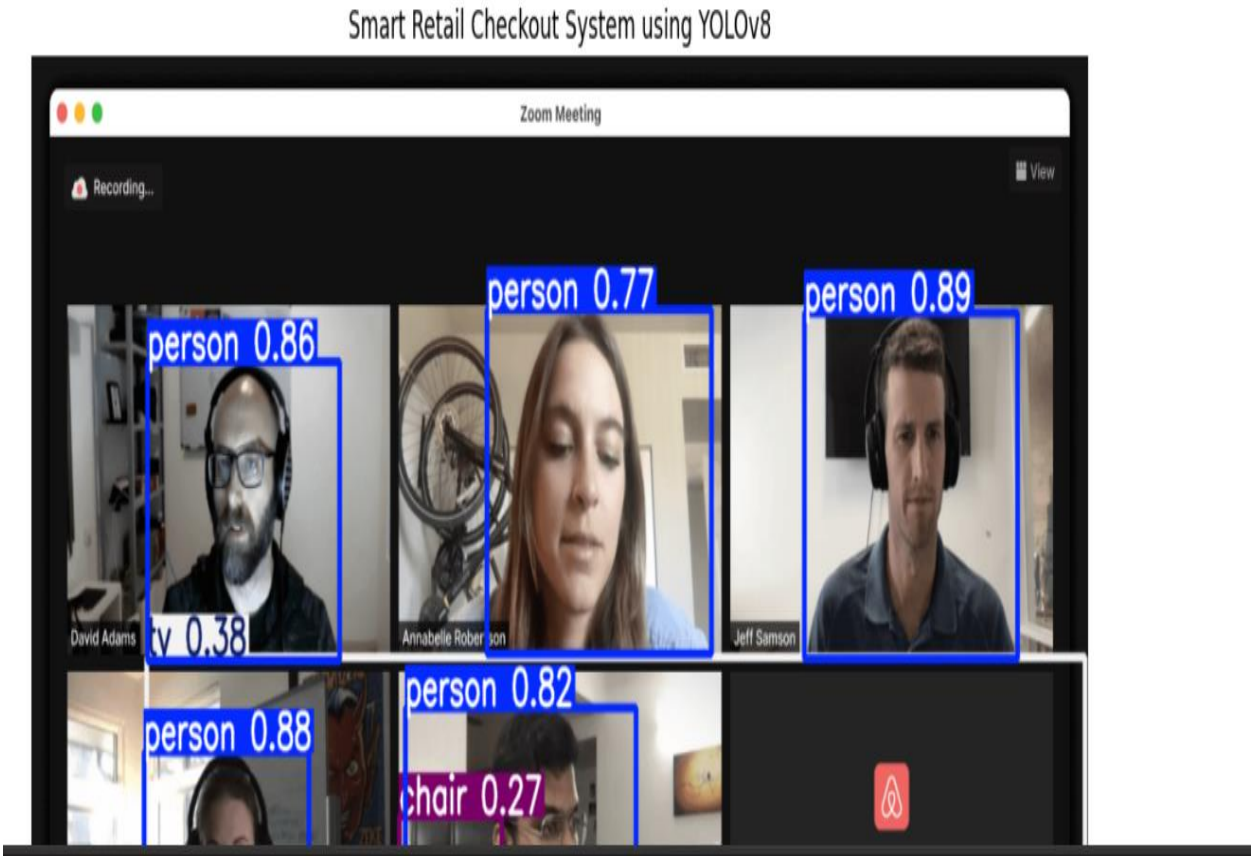
```
Video saved as output.mp4
MoviePy - Building file output.gif with imageio.

GIF saved as output.gif
```

Case Study 2: Retail Checkout Automation Object recognition helps in building smart checkout systems by identifying items on a counter, reducing human effort and errors.

```
# Case Study 2: Retail Checkout Automation
from ultralytics import YOLO
import cv2
import matplotlib.pyplot as plt
model = YOLO('yolov8n.pt')
image_path = "D:/23108111/H22.png"
image = cv2.imread(image_path)
results = model(image_path)
annotated_img = results[0].plot()
img_rgb = cv2.cvtColor(annotated_img, cv2.COLOR_BGR2RGB)
plt.figure(figsize=(12, 8))
plt.imshow(img_rgb)
plt.axis('off')
plt.title("Smart Retail Checkout System using YOLOv8")
plt.show()
```

```
image 1/1 D:\23108111\H22.png: 416x640 5 persons, 1 chair, 1 tv, 95.0ms
Speed: 4.4ms preprocess, 95.0ms inference, 1.3ms postprocess per image at shape (1, 3, 416, 640)
```



Case Study 1: Smart Classroom Monitoring

Scenario: An educational institution wants to automate attendance tracking in smart classrooms using object detection and facial recognition. Cameras installed in classrooms should detect students, recognize faces, and log attendance automatically.

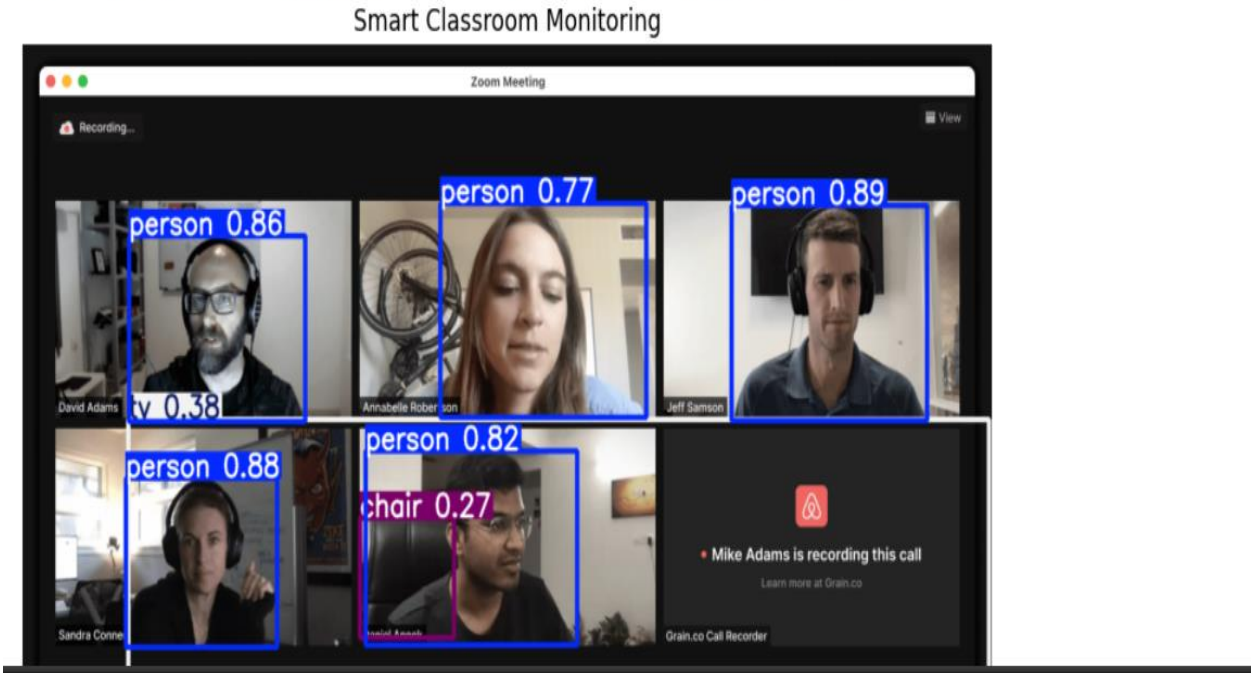
Objectives:

- **Detect human presence using object detection (YOLO or Haar cascade).**
- **Use face recognition to identify students.**
- **Log recognized faces with timestamps.**

Technologies: YOLOv8 for person detection, OpenCV Face Recognition API, SQLite for logging.

```
# Case Study 1: Smart Classroom Monitoring
from ultralytics import YOLO
import cv2
import matplotlib.pyplot as plt
model = YOLO("yolov8n.pt")
image_path = "D:/23108111/H22.png"
image = cv2.imread(image_path)
results = model(image_path)
annotated = results[0].plot()
img_rgb = cv2.cvtColor(annotated, cv2.COLOR_BGR2RGB)
plt.figure(figsize=(10, 6))
plt.imshow(img_rgb)
plt.axis("off")
plt.title("Smart Classroom Monitoring")
plt.show()
```

```
image 1/1 D:\23108111\H22.png: 416x640 5 persons, 1 chair, 1 tv, 79.1ms
Speed: 3.7ms preprocess, 79.1ms inference, 1.2ms postprocess per image at shape (1, 3, 416, 640)
```



Case Study 2: Smart Parking Surveillance System

Scenario: A smart city initiative aims to automate parking lot monitoring. The system should detect vehicles entering and exiting the lot, recognize license plates, and update availability in real-time.

Objectives:

- **Detect cars, bikes, and trucks using object detection.**
- **Apply OCR (Optical Character Recognition) to recognize license plates.**
- **Update parking slot availability.**

Technologies: YOLO for vehicle detection, Tesseract OCR for license plates, OpenCV.

```
#Case Study 2: Smart Parking Surveillance System
from ultralytics import YOLO
import cv2
import matplotlib.pyplot as plt
model = YOLO("yolov8n.pt")
image_path = "D:/23108111/H19.jpg"
image = cv2.imread(image_path)
results = model(image_path)
annotated = results[0].plot()
img_rgb = cv2.cvtColor(annotated, cv2.COLOR_BGR2RGB)
plt.figure(figsize=(10, 6))
plt.imshow(img_rgb)
plt.axis("off")
plt.title("Smart Parking Surveillance")
plt.show()
```

```
image 1/1 D:\23108111\H19.jpg: 480x640 2 persons, 1 car, 3 buss, 1 truck, 69.0ms
Speed: 4.8ms preprocess, 69.0ms inference, 1.5ms postprocess per image at shape (1, 3, 480, 640)
```

Case Study 3: Wildlife Conservation Monitoring

Scenario: A wildlife reserve deploys camera traps to monitor animal activity. The system should automatically detect and recognize different animal species from captured images or video clips.

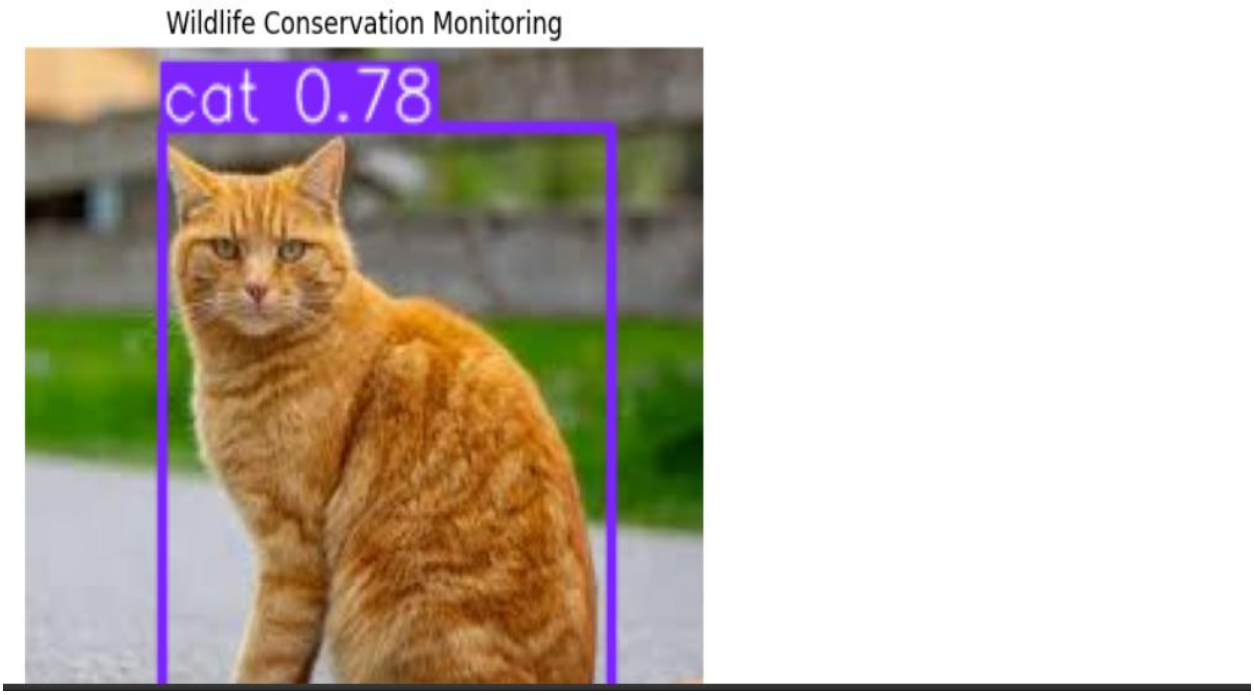
Objectives:

- **Detect animals in the wild using YOLO models trained on animal datasets.**
- **Classify species such as tiger, deer, elephant, etc.**
- **Generate daily wildlife reports with timestamps.**

Technologies: YOLOv8 trained on wildlife datasets, TensorFlow/Keras for classification, OpenCV.

```python
# Case Study 3: Wildlife Conservation Monitoring
from ultralytics import YOLO
import cv2
import matplotlib.pyplot as plt
model = YOLO("yolov8n.pt")
image_path = "D:/23108111/H29.jpg"
image = cv2.imread(image_path)
results = model(image_path)
annotated = results[0].plot()
img_rgb = cv2.cvtColor(annotated, cv2.COLOR_BGR2RGB)
plt.figure(figsize=(10, 6))
plt.imshow(img_rgb)
plt.axis("off")
plt.title("Wildlife Conservation Monitoring")
plt.show()
```

```
image 1/1 D:\23108111\H29.jpg: 640x640 1 cat, 125.0ms
Speed: 6.0ms preprocess, 125.0ms inference, 1.4ms postprocess per image at shape (1, 3, 640, 640)
```



Case Study 4: Industrial Defect Detection System

Scenario: A textile manufacturing unit wants to detect defective products on a conveyor belt. The system should identify tears, stains, or misaligned patterns using object detection.

Objectives:

- **Detect defects using trained object detection models.**
- **Flag defective items for removal.**
- **Collect statistics on defect frequency for quality control.**

Technologies: YOLO custom model, OpenCV for image capture and processing.

```
# Case Study 4: Industrial Defect Detection System
from ultralytics import YOLO
import cv2
import matplotlib.pyplot as plt
model = YOLO("yolov8n.pt")
image_path = "D:/23108111/H21.jpg"
image = cv2.imread(image_path)
results = model(image_path)
annotated = results[0].plot()
img_rgb = cv2.cvtColor(annotated, cv2.COLOR_BGR2RGB)
plt.figure(figsize=(10, 6))
plt.imshow(img_rgb)
plt.axis("off")
plt.title("Industrial Defect Detection")
plt.show()
```

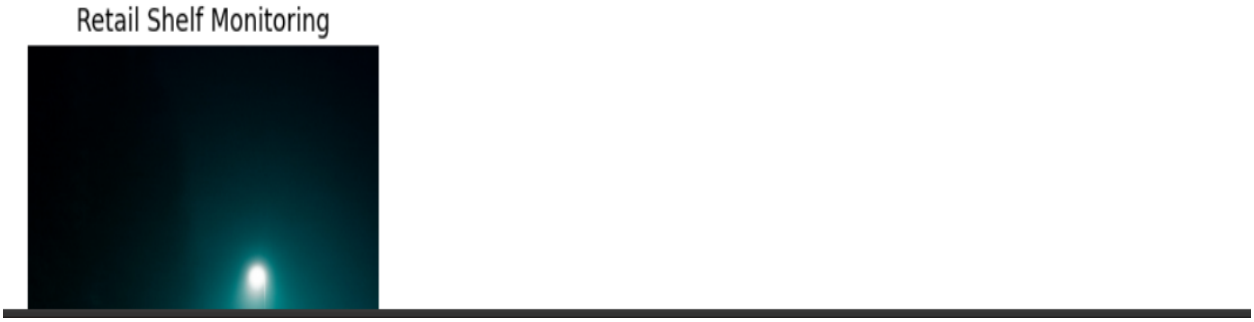
```
image 1/1 D:\23108111\H21.jpg: 640x480 1 microwave, 1 oven, 115.7ms
Speed: 2.6ms preprocess, 115.7ms inference, 1.2ms postprocess per image at shape (1, 3, 640, 480)
```

Case Study 5: Retail Store Analytics with Shelf Monitoring
Scenario: A retail chain installs smart cameras in stores to monitor shelf stock. The system should detect product presence, identify empty shelves, and trigger restocking alerts.
Objectives:
□ Detect and count product items on shelves.
□ Recognize missing or misplaced items.
□ Generate visual reports for store managers.

Technologies: YOLOv8 for object detection, OpenCV for video processing, dashboard for visualization.

```
# Case Study 5: Retail Store Analytics with Shelf Monitoring
from ultralytics import YOLO
import cv2
import matplotlib.pyplot as plt
from collections import Counter
model = YOLO("yolov8n.pt")
image_path = "D:/23108111/H9.jpg"
image = cv2.imread(image_path)
results = model(image_path)
annotated = results[0].plot()
items = []
for r in results:
    for box in r.boxes:
        cls = int(box.cls[0])
        items.append(model.names[cls])
count = Counter(items)
print("Shelf Inventory:")
for item, qty in count.items():
    print(item, ":", qty)
img_rgb = cv2.cvtColor(annotated, cv2.COLOR_BGR2RGB)
plt.figure(figsize=(10, 6))
plt.imshow(img_rgb)
plt.axis("off")
plt.title("Retail Shelf Monitoring")
plt.show()
```

```
image 1/1 D:\23108111\H9.jpg: 640x384 2 traffic lights, 79.8ms
Speed: 2.5ms preprocess, 79.8ms inference, 1.1ms postprocess per image at shape (1, 3, 640, 384)
Shelf Inventory:
traffic light : 2
```



https://github.com/Habiba-Bibi/